MetaRocq

(define-syntax (MetaRocq stx)
#''(🪨s all the way down))

The MetaRocq project

MetaRocq builds on top of **Template-Rocq**.

Template-Rocq is a quoting library that takes Rocq terms and constructs a representation of their syntax tree as an inductive data type.

MetaRocq is a project formalizing Rocq in Rocq and provides tools for:

- Manipulating Rocq terms
- Developing certified plugins in Rocq

Example plugins include:

- Translations
- Compilers
- Tactics

The goal of the MetaRocq project is to provide a self-hosted, bootstrappable and verifiable implementation of Rocqs unification and type-checking algorithms.

MetaRocq can also **be used** to build higher-level tools. E.g. translations, boilerplate generators, domain-specific proof languages, or general purpose tactic languages.

MetaRocq allows users to define many kinds of general purpose plugins, whose correctness can be readily proved in the system itself, and can be run efficiently after extraction.

A small example

```
Inductive tm : Set :=
| var : nat → tm
| lam : tm → tm
| app : tm → tm → tm.

Run TemplateProgram (add_constructor
<% tm %> "letin"
<% fun tm' ⇒ tm' → tm' → tm' %>).

Inductive tm' : Set :=
| var' : nat → tm'
| lam' : tm' → tm'
| app' : tm' → tm' → tm'
| letin : tm' → tm' → tm'.
```

This command has the same effect as declaring the inductive tm' by hand, but with the benefit that if tm is changed, then tm' is automatically changed accordingly.

It is not possible to define such a transformation using the tactic language of Coq.

We can however, define a dedicated plugin, but that is tedious at best.

Using MetaCoq, we can work directly in Coq, with a standardized API which isn't subject to implementation changes in the ML code of Coq.